Hermetic protocol-promotion research — pure-python peer servers over a kernel-WireGuard netns tunnel

Revision: 5 **Last modified:** 2026-07-02T07:49:30Z **Status:** Research findings (deep multi-angle web research per §11.4.150 / §11.4.99). No code changed by the research pass. Authority: inherits `constitution/Constitution.md` per §11.4.35. **Scope:** strengthen the hermetic WireGuard test harness so operator-gated VPN-LAN protocol round-trips (Chromecast/DIAL, FTP, WebDAV, SMB, mDNS, SMTP/IMAP) can be promoted to run AUTONOMOUSLY inside unprivileged `unshare -Urnm` user+net namespaces joined by a real kernel-WireGuard tunnel, with pure-python-stdlib peer servers bound to the WG overlay address.

Harness context this research feeds

The committed harnesses (`tests/vpn_lan/hermetic_wg_roundtrip.sh`, `hermetic_bridge_run.sh`, `hermetic_ftp_run.sh`) prove a real payload round-trips over an encrypted kernel-WireGuard tunnel between two unprivileged network namespaces, fully rootless:

```
 netns A (in the userns)                    netns B (peer /
holder)
 veth0 10.9.0.1  ── underlay (UDP) ─── veth1 10.9.0.2     <-
carries encrypted WG
 wg0   10.10.0.1 ══ WireGuard tunnel ═══ wg0 10.10.0.2  <- the
overlay
                                            python3 peer server
bound to 10.10.0.2
```

The peer server binds the WG-only overlay `10.10.0.2` (allowed-ips `10.10.0.2/32`), reachable ONLY through the tunnel; independent oracles gate PASS (payload round-trip + `wg show` handshake with `rx>0 && tx>0`); a per-layer golden-bad (`WG_MUT=badkey`, `H2_MUT=badeureka`, `FT_MUT=empty`) is the §1.1 control. The live protocol tests (SMB/NFS/FTP/SFTP/WebDAV/IMAP/SMTP/POP3S/Cast/DIAL/ADB) are AUTHORED but SKIP-gated on the operator's

svord/Mullvad bridge. This research is about replacing the "need a live remote server" gate with a pure-stdlib peer server inside netns B.

1. Pure-python-stdlib FTP server correctness in a network namespace

RFC 959 §4.1.2: the `227 Entering Passive Mode (h1,h2,h3,h4,p1,p2)` reply encodes the **host address the server is listening on** (h1 = high octet) and the data port as `p1*256 + p2`. The load-bearing gotcha: a naïve server advertises `127,0,0,1` (or the underlay), so the client dials an unreachable address and the data channel hangs — the classic FTP-in-NAT/namespace bug (vsftpd fixes it with `pasv_address`). For our harness the reachable address is the WG overlay `10.10.0.2`; the server MUST encode `10,10,0,2` in the 227 reply. **EPSV** (`229 Entering Extended Passive Mode (|||port|)`, RFC 2428) sidesteps the IP-encoding entirely — it carries no address, so the client reuses the control connection's peer (`10.10.0.2` over the tunnel); curl prefers EPSV. Because the overlay is `allowed-ips 10.10.0.2/32`, a successful data connection is itself proof of tunnel traversal (the client cannot reach `10.10.0.2` any other way).

Minimal command set `curl --ftp-pasv` drives for LIST + RETR: `USER/PASS/SYST/PWD/TYPE/(EPSV|PASV)/LIST/RETR/QUIT`. Python has NO stdlib FTP *server* (only the `ftplib` client), so a ~85–120-line `socket`-based server is required — no `pyftpdlib` install (§11.4.122). **Implemented** in `hermetic_ftp_run.sh` (EPSV/PASV both advertise `10.10.0.2`; golden-bad `FT_MUT=empty` serves an empty listing → the promoted test SKIPs, proving a real PASS needs a real non-empty listing).

Gotcha found in review: the `LIST` output is CRLF-terminated; the promoted test's `awk 'NF{print $NF}'` leaves a trailing `\r` on the filename, and curl rejects a URL with an embedded `\r` (CVE-2014-8150 → `CURLE_URL_MALFORMAT`). The promoted test's fetch is best-effort and NOT part of its PASS gate, so this is harmless there — but the harness must prove the byte-transfer half itself: `hermetic_ftp_run.sh` RETRs the nonce file by its exact (self-constructed, CRLF-free) name and byte-compares to the known payload (§11.4.107(9)).

2. WebDAV origin + forward proxy in pure python stdlib

`http.server.BaseHTTPRequestHandler` dispatches by method name → define `do_PROPFIND`, `do_OPTIONS`, `do_GET`, `do_PUT`. RFC 4918 §9.1: PROPFIND with `Depth: 0|1`, response `207 Multi-Status`, `Content-`

`Type: application/xml`, body rooted at `DAV:multistatus` with per-resource `DAV:response` → `DAV:href` + `DAV:propstat`(`DAV:prop` + `DAV:status`). Minimal valid body:

```
<?xml version="1.0" encoding="utf-8"?>
<D:multistatus xmlns:D="DAV:">
  <D:response>
    <D:href>/dav/file.txt</D:href>
    <D:propstat>
      <D:prop><D:displayname>file.txt</D:displayname>
             <D:getcontentlength>12</D:getcontentlength>
             <D:resourcetype/></D:prop>
      <D:status>HTTP/1.1 200 OK</D:status>
    </D:propstat>
  </D:response>
</D:multistatus>
```

Forward proxy (RFC 7230 §5.3.2): a proxy MUST accept **absolute-form** request targets; `curl -x http://proxy http://upstream/path` sends `PROPFIND http://upstream/path HTTP/1.1` to the proxy, which parses the absolute-URI, connects upstream, and **re-emits origin-form** (`PROPFIND /path HTTP/1.1` + a fresh `Host:` from the request-target, §5.3.1). Both origin and a ~40-line stdlib forward proxy are feasible. The promoted WebDAV leg (`ftp_sftp_webdav.sh`) routes through `HELIX_SQUID_PROXY` and expects `207` (non-207 ⇒ FAIL, fail-closed §11.4.68) — so the hermetic promotion needs BOTH a `207`-returning origin at `10.10.0.2` and a client-side forward proxy the curl `-x` routes through. Golden-bad `WEBDAV_MUT=plain200` (origin returns `200`) MUST trip the test's fail-closed non-207 branch.

3. Kernel-WireGuard in an unprivileged user namespace — caveats

Works because `CLONE_NEWUSER + CLONE_NEWNET` grants `CAP_NET_ADMIN` **inside** the new userns, so `ip link add wg0 type wireguard` + `wg set` succeed with the host `wireguard` module loaded — no `wireguard-go`, no host root. `ip netns add` still needs root (persists under `/run/netns/`); `unshare -Urn` is the rootless path but namespaces are process-lifetime, non-persistent, invisible to `ip netns` (our harness relies on this).

The real risk is portability, not a WG-specific CVE. Independent research (Edera, kernel 6.18): creating virtual net devices auto-loads 15+ kernel modules unprivileged; 40+ kernel CVEs (2020–2025) had userns as a prerequisite; userns raised reachable sensitive kernel operations 8/40 → 27/40 (~262%). Hardening: `kernel.unprivileged_userns_clone=0` AND `user.max_user_namespaces=0`. **Ubuntu 23.10+ restricts unprivileged userns via AppArmor**

(`kernel.apparmor_restrict_unprivileged_userns=1`): an unconfined process CANNOT create a userns without a profile carrying `userns,` — so a tool invoking `unshare -Urn` from an arbitrary path simply fails on such hosts. **Actionable:** keep the honest-SKIP taxonomy first-class — probe + SKIP-with-reason separately on (a) `/sys/module/wireguard` absent, (b) `unshare -Urnm true` denied, (c) `ip link add type wireguard` failing — so a hardened CI host SKIPs honestly, never false-greens. Our harnesses already SKIP on (a) and (b); (c) is covered by the inner `|| fail`.

4. OSS / tools for rootless hermetic network testing (§11.4.74)

Tool	What	Root/ install	Relevance
`unshare -Urn` + kernel WG (current)	rootless userns + in-kernel WG	none; needs host WG module + userns permitted	simplest, real encryption; fragile on userns-hardened hosts
slirp4netns	user-mode TCP/IP stack for a netns	none; small binary	netns egress w/o veth/ CAP_NET_ADMIN; NAT overhead
pasta / passt	newer user-mode net; Podman rootless default	none; small binary	faster than slirp4netns (no NAT); best rootless egress
wireguard-go	compliant userspace WG (Go, TUN)	Go binary; `/dev/net/tun`	kernel-module fallback when WG module absent
boringtun	Cloudflare userspace WG (Rust)	binary; drop-priv caveat	same fallback, faster; Rust binary dep
wireguard4netns (CMU)	patched wireguard-go for unprivileged netns	none	closest art to "hermetic WG w/o kernel module"; reuse per §11.4.74
gVisor netstack	userspace TCP/IP as a Go lib	Go build	fully in-process encrypted stack (no TUN/netns/ module); future direction

Tool	What	Root/ install	Relevance
rootless podman + `vasic-digital/containers`	container fixtures	rootless podman; §11.4.76	for SMB/NFS/TLS-mail: boot a real server container in netns B rather than hand-roll

Per-protocol promotion verdict (autonomous, no host install): FTP, WebDAV, Cast/DIAL eureka, plaintext SMTP/IMAP/POP3, HTTP-family → pure-stdlib peer feasible now. Implicit-TLS mail (465/993/995) → stdlib `ssl` + a self-signed cert generated at test time (bounded dep for cert-gen). mDNS/DNS-SD → pure-python multicast responder feasible (`socket` + `IP_ADD_MEMBERSHIP` on `224.0.0.251:5353`) — but bind the overlay addr, not localhost (python-zeroconf caveat), and note real cross-subnet mDNS still needs a reflector in production. **SMB, NFS → no faithful pure-stdlib server; use a container-backed peer via `vasic-digital/containers` (§11.4.76) or stay operator-gated. Do NOT hand-roll SMB/NFS.**

5. Anti-bluff verification patterns (the oracle problem)

Determining "did the round-trip really happen over the encrypted tunnel vs leak over the underlay" is a test-oracle problem; **metamorphic testing** (checking relations between related runs) is the standard escape when there is no golden output. Our stack maps to it:

1. **Byte/sha256 payload verification** (own-content golden source) — strongest oracle when we control the payload.
2. **Wrong-key negative control** (`WG_MUT=badkey`) — a metamorphic relation with negation (corrupt key → PASS transitions to hard-FAIL); doubles as §11.4.107(10) self-validated golden-bad. Replicate per protocol.
3. **Reachability-as-proof** — `allowed-ips 10.10.0.2/32` means a successful L4 connection is itself tunnel-traversal evidence. Strengthen with a wrong-destination negative control (same fetch to underlay `10.9.0.2` / wg0-down MUST fail).
4. **Transport-counter cross-check** (`wg show` handshake + `rx/tx`) — an oracle in a different domain than the payload check (§11.4.107(2) multi-oracle).
5. **Underlay-sniff differential** (optional, strongest) — capture on the underlay veth (`AF_PACKET`/`tcpdump`) and assert the plaintext nonce never appears in cleartext on the underlay while ciphertext flows. Gate behind an honest SKIP where unavailable.

Precedent (§11.4.8): mature literature exists for metamorphic testing / the oracle problem generally (Wikipedia; arXiv 1912.05278 security-flavoured MT), but **no external precedent covers the exact composite** — "prove a plaintext payload round-tripped over a rootless kernel-WireGuard netns tunnel and did not leak on the underlay, via self-fetched nonce + wg-transport-counter + wrong-key golden-bad." That specific composite is **original work**, well-grounded in the general MT/oracle literature.

6. Pure-python-stdlib implicit-TLS mail (SMTPS/IMAPS/POP3S) round-trip — H2.email

Promoting `email_roundtrip.sh` autonomously requires a mail peer bound to the WG-only overlay `10.10.0.2` speaking implicit-TLS SMTP/IMAP/POP3. Findings that shaped the `hermetic_email_run.sh` design:

1. **Python 3.12 removed the `smtpd` module (PEP 594 "dead batteries").** The stdlib no longer ships an SMTP *server*; the *clients* (`smtplib`, `imaplib`, `poplib`) remain. The hermetic peer therefore hand-rolls a minimal SMTP/IMAP/POP3 server directly on `socket` + `ssl` — no third-party install (§11.4.74 catalogue-check: no reusable stdlib server exists post-3.12; original minimal server is the correct path).
2. **Implicit TLS ("SMTPS/IMAPS/POP3S", ports 465/993/995) wraps the whole connection in TLS from byte 0**, per RFC 8314 (which recommends implicit TLS over STARTTLS). `ssl.SSLContext(PROTOCOL_TLS_SERVER).wrap_socket(server_side=True)` with a self-signed cert generated per-run (mktemp, mode 0600, never logged — §11.4.10) is sufficient for a hermetic peer; the client verifies against that per-run cert, not a CA.
3. **CRITICAL load-bearing FACT (§11.4.6, captured in the de-risk PoC): the TLS server MUST perform a clean bidirectional close — send `close_notify` (call `SSLSocket.unwrap()` before `close()`).** A truncated TLS close makes `openssl s_client` (and stricter clients) exit non-zero, which would silently drop the promoted leg to SKIP instead of PASS — a false-negative that masquerades as an honest skip. This is the single most important hermetic-mail implementation detail; the harness verifies a clean close per leg.
4. **Command grammar from the standards:** SMTP `EHLO`/`AUTH LOGIN`(base64)/`MAIL FROM`/`RCPT TO`/`DATA` (RFC 5321 + RFC 4954 AUTH); IMAP `LOGIN`/`LIST` (RFC 3501); POP3 `USER`/`PASS`/`RETR` (RFC 1939). Each leg's PASS is gated on the protocol's own success signal (SMTP 2xx end-of-DATA, IMAP `* LIST` untagged response, POP3 `+OK` + retrieved body), never a mere TCP connect.

5. **Round-trip oracle + two golden-bad teeth (§11.4.107(10)).** SMTP `DATA` embeds a per-run nonce token; the round-trip PASSes only when POP3S `RETR` returns a body containing that exact token (own-content golden source, strongest oracle). Two independent mutations each prove a *distinct* assertion is load-bearing: `MAIL_MUT= openrelay` (peer accepts an external-domain `RCPT` → the open-relay negative control MUST flip to FAIL) and `MAIL_MUT=droptoken` (peer delivers a mailbox WITHOUT the sent token → the round-trip MUST FAIL). One tooth guarding the security-negative, one guarding the round-trip-positive — neither can be satisfied by a bluff peer.

Precedent (§11.4.8): the RFCs + PEP 594 fully specify the protocol surface; no external project provides "a pure-stdlib implicit-TLS SMTP+IMAP+POP3 peer bound to a kernel-WG netns overlay with an open-relay + drop-token golden-bad pair." That composite is **original work**, grounded in the cited standards.

Three most actionable findings for the next iterations

1. **Promote FTP + WebDAV first with pure-stdlib peers bound to `10.10.0.2` — the 227-reply IP (or EPSV) is the load-bearing correctness rule.** FTP done (`hermetic_ftp_run.sh`); WebDAV needs a `207`-returning `do_PROPFIND` origin + a client-side forward proxy the curl `-x` routes through.
2. **Make the userns/kernel-WG environment dependency an explicit greppable SKIP taxonomy** — Ubuntu 23.10+/ hardened kernels deny `unshare -Urn`; separate SKIP reasons prevent a hardened host from false-greening. Note wireguard-go/boringtun/wireguard4netns/ gVisor-netstack as the no-kernel-module fallback (build cost accepted only then).
3. **Replicate the `WG_MUT=badkey` golden-bad per protocol + add a wrong-destination (underlay) negative control**, and an optional `AF_PACKET` underlay-sniff differential as the rock-solid non-leak proof (§11.4.123).

7. Underlay-sniff differential — the rock-solid non-leak proof (IMPLEMENTED on the WG substrate — 91af9c6)

Status (§11.4.6): **IMPLEMENTED** in `hermetic_wg_roundtrip.sh` (commit `91af9c6`, task #63) — the substrate harness now carries the sniff; the design below is what shipped. Independent

§11.4.142 review `a2b3c696` returned **GO** on 11/11 checks against real runs: `SNIFF_MUT=plain` flips ONLY assertion (b) "plaintext absent" to FAIL 3/3 while ciphertext stays present (load-bearing, not a tautology, §11.4.107(10)); NORMAL → `ciphertext(0x04 :51820)=present plaintext_nonce=absent` 3/3; header-only pcap → analyzer exit 3 (honest FAIL); forced-iface + no-tcpdump → honest `SNIFF-SKIP`; `WG_MUT=badkey` undisturbed (sniff sits past the `UP!=1` gate); veth0-only capture, 3.5 s / 4 MB / `timeout`-bounded, `SNIFF_PID` reaped, `wg0-mullvad` untouched (§11.4.174); `sh -n` + `bash -n` clean. Follow-ups: **(#64) LANDED** (`cdb0ccd`, independent §11.4.142 review `af9f67ad` GO 5/5) — a one-line Ethernet ethertype guard on the frame analyzer (VLAN-tag offset shift, structurally impossible on the harness's own fresh veth, correct-by-construction; proven load-bearing by a guard-removal self-test — WITH guard `vlan_ct=absent` PASS, WITHOUT `vlan_ct=present` FAIL); **(#65)** fan the differential out to the protocol harnesses — the design pass (§7.1 below) found it meaningful for **3** of them (bridge/ftp/webdav, all plaintext-under-WG) but **N/A for email** (implicit-TLS encrypts the token *below* WG, so "plaintext absent on the underlay" is tautologically true regardless of the tunnel).

It was the strongest hardening left for the hermetic stream (FINDINGS §5 point 5), and the natural successor to the wrong-destination negative control (task #62): that control proves the peer service is *reachable* only via the overlay; the underlay-sniff differential proves the *payload actually travelled encrypted* and never leaked in cleartext on the wire.

The gap it closes. Today's oracles are (a) reachability-as-proof (overlay-only bind + underlay-destination negative), (b) `wg show` handshake/rx-tx counters, (c) the `WG_MUT=badkey` golden-bad (bad key ⇒ no traversal). None of them *reads the bytes on the underlay* to confirm the plaintext nonce is absent while ciphertext flows — a different-domain oracle (§11.4.107(2)) and the canonical WireGuard self-audit method.

Design. During the positive round-trip, in the CLIENT netns, capture on the **underlay** veth (the interface carrying the encrypted WG UDP, `10.9.0.x`) for the fetch window, then assert BOTH:

1. **Ciphertext present** — ≥1 UDP datagram to the WG listen port whose payload begins with the WireGuard transport-data type prefix `0x04 00 00 00` (type-4 data message; handshake init/resp are types 1/2). Proves the tunnel actually carried traffic on the underlay.
2. **Plaintext absent** — the per-run nonce/token (the same fresh value the §11.4.107 not-stale self-fetch already uses) does **NOT** appear anywhere in the raw captured underlay bytes. Proves the payload was encrypted, not leaked.

Feasibility (rootless, verified by research §11.4.99). An `AF_PACKET` raw socket requires `CAP_NET_RAW` *in the user namespace governing the netns* — which the harness's `unshare -Urnm` root-in-userns already holds for its OWN veth, so the capture runs with **zero host privilege / no install** (matches the whole hermetic design). Two implementation paths: `tcpdump -i <veth> -nn -w <pcap>` if present (SKIP-with-reason §11.4.3 if not), or a pure-python `socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL))` bound to the veth (no dependency — the same zero-install stance as the stdlib peers). Capture to a bounded buffer / short `timeout` window (§12.6), teardown in the existing `trap`.

Anti-bluff golden-bad (§11.4.107(10)) — this analyzer MUST be self-validated. A sniff that only ever runs against the encrypted path is a tautology (the nonce is *never* there). The load-bearing control: a golden-bad mode that routes the SAME fetch over a **plaintext** path on the underlay (e.g. fetch the peer's *underlay* HTTP directly, or `wg0` down + peer temporarily bound on `10.9.0.x`) so the nonce **DOES** appear in the captured cleartext → the "plaintext absent" assertion MUST then FAIL. Only if the golden-bad makes it fail is the sniff proven to gate anything. Pairs with a §1.1 meta-mutation.

Honest boundary (§11.4.6). This proves non-leak on the LOCAL simulated underlay veth, NOT on the real Mullvad WAN path (that stays the §11.4.3 operator-gated confirmation). `AF_PACKET` in a userns has documented kernel-attack-surface history (Project Zero packet-socket CVEs) — we only *read* our own netns's veth, never the host's interfaces (§11.4.174). Scope it as ONE harness leg (start with `hermetic_wg_roundtrip.sh`, the substrate) before fanning out.

7.1 Fan-out design (task #65) — which harnesses genuinely warrant the sniff (§11.4.150)

Design pass (subagent `a314c3e5`, read-only, file:line-cited against each harness). The substrate pattern (`hermetic_wg_roundtrip.sh:149-304`) clones onto a protocol harness by: capture on the client underlay `veth0` **before** the harness's existing not-stale self-fetch, assert (a) a WG type-4 (`0x04`) datagram to `:51820` present and (b) the per-run plaintext marker absent from the raw underlay bytes; `SNIFF_MUT=plain` emits that marker as cleartext UDP to discard `:9` (distinct from every harness's TCP negative-control port) so ONLY assertion (b) flips.

Harness	Plaintext marker	Rides plaintext-under-WG?	Sniff meaningful?	Injection
`hermetic_bridge_run.sh`	`$DEV_NAME` in eureka JSON `name`	YES (plain HTTP)	**YES — build**	`$DEV_NAME` → `10.9.0.2:9`
`hermetic_ftp_run.sh`	`$NONCE` (LIST filename + RETR body)	YES (FTP cleartext)	**YES — build**	`$NONCE` → `10.9.0.2:9`
`hermetic_webdav_run.sh`	`$NONCE` in 207 etag (proxy→origin `10.10.0.2:8080` hop rides `wg0`; client→proxy is loopback, irrelevant)	YES (plain HTTP 207)	**YES — build**	`$NONCE` → `10.9.0.2:9`
`hermetic_email_run.sh`	round-trip token in SMTP DATA → POP3S RETR	**NO** — implicit-TLS wraps the payload (`:170/:356`, client `openssl s_client :413`) **before** WG	**NO — do NOT build (vacuous)**	—

Honest verdict (§11.4.6). Build the sniff on **bridge/ftp/webdav** — each app payload is cleartext, so the marker rides *plaintext-under-WG* and the differential genuinely exercises WG's protection (the `SNIFF_MUT=plain` golden-bad is load-bearing). Do **NOT** build it on **email**: the token is TLS-encrypted below WG, so "plaintext token absent on the underlay" is trivially true *even if WG were absent* — a green sniff there would be green for the TLS reason, not the WG reason (a §11.4-forbidden vacuous test that a `SNIFF_MUT=plain` tooth cannot rescue). Email's tunnel-gating is already proven by the overlay-only bind + §11.4.111 wrong-destination negative + `wg show` rx/tx counters; the correct action is an honest documented N/A note, not a fourth sniff. The email TLS-under-WG vacuity verdict is **original analysis** derived directly from the harness source (no external precedent needed); the sniff pattern itself is the GO-reviewed substrate (§7).

Sources verified

All URLs accessed **2026-07-02**.

- RFC 959 (FTP — PASV/227, TYPE, RETR/LIST): https://www.rfc-editor.org/rfc/rfc959
- RFC 2428 (FTP EPSV/EPRT): https://www.rfc-editor.org/rfc/rfc2428
- D. J. Bernstein, "The PASV, RETR, REST, and PORT verbs": https://cr.yp.to/ftp/retr.html
- IBM support — internal IP in 227 reply: https://www.ibm.com/support/pages/remote-ftp-client-doing-passive-mode-gets-internal-ip-address-returned-227-entering-passive-mode-message
- oneuptime — FTP passive mode troubleshooting: https://oneuptime.com/blog/post/2026-03-20-ftp-passive-mode-troubleshoot-ipv4/view
- Cerberus FTP support — passive data connection failures: https://support.cerberusftp.com/hc/en-us/articles/202639399
- Check Point sk169993 — 227 header IP vs command IP mismatch: https://support.checkpoint.com/results/sk/sk169993
- everything.curl.dev — FTP directory listing (CRLF): https://everything.curl.dev/ftp/dirlist.html
- curl(1) man page: https://curl.se/docs/manpage.html
- libcurl error codes (CURLE_URL_MALFORMAT): https://curl.se/libcurl/c/libcurl-errors.html
- Debian LTS — CVE-2014-8150 URL sanitization: https://lists.debian.org/debian-lts/2015/01/msg00012.html
- MicroPython FTPdLite — minimalist RFC-959 asyncio FTP server: https://github.com/orgs/micropython/discussions/13016
- Python stdlib `http.server` docs: https://docs.python.org/3/library/http.server.html
- PEP 268 — Extended HTTP functionality and WebDAV: https://peps.python.org/pep-0268/
- RFC 4918 (WebDAV — PROPFIND, 207 Multi-Status): https://www.rfc-editor.org/rfc/rfc4918
- sourceperl gist — WebDAV PROPFIND in python: https://gist.github.com/sourceperl/8d50995a5265f84fa5ea1dd4adf7522f
- RFC 7230 (§5.3.1 origin-form, §5.3.2 absolute-form/proxy): https://www.rfc-editor.org/rfc/rfc7230.html
- curl issue #6769 — proxy request absolute-form vs RFC 7230: https://github.com/curl/curl/issues/6769
- Edera — "User namespaces are not a security boundary": https://edera.dev/stories/user-namespaces-are-not-a-security-boundary
- Ubuntu — "Ubuntu 23.10 restricted unprivileged user namespaces": https://ubuntu.com/blog/ubuntu-23-10-restricted-unprivileged-user-namespaces

- Ubuntu bug 2046477 — enable userns restrictions by default: https://bugs.launchpad.net/ubuntu/+source/apparmor/+bug/2046477
- systemshardening.com — Restricting Unprivileged User Namespaces: https://www.systemshardening.com/articles/linux/linux-unprivileged-namespace-restriction/
- Kicksecure — User Namespace hardening: https://www.kicksecure.com/wiki/User_Namespace
- Red Hat — "Use a net namespace for VPNs": https://www.redhat.com/en/blog/use-net-namespace-vpn
- blog.0x1b.me — Unprivileged Linux Network Namespaces Pt 1: https://blog.0x1b.me/posts/unprivileged-linux-netns-pt1/
- benjamintoll — On Unsharing Namespaces Pt 2: https://benjamintoll.com/2022/12/14/on-unsharing-namespaces-part-two/
- PEP 594 — Removing dead batteries from the stdlib (`smtpd` removed in 3.12): https://peps.python.org/pep-0594/
- Python stdlib `ssl` docs (`wrap_socket`, `unwrap`/close_notify): https://docs.python.org/3/library/ssl.html
- Python stdlib `smtplib` / `imaplib` / `poplib` client docs: https://docs.python.org/3/library/smtplib.html
- RFC 8314 (implicit TLS for email submission/access — ports 465/993/995): https://www.rfc-editor.org/rfc/rfc8314
- RFC 5321 (SMTP — EHLO/MAIL/RCPT/DATA): https://www.rfc-editor.org/rfc/rfc5321
- RFC 4954 (SMTP AUTH — AUTH LOGIN): https://www.rfc-editor.org/rfc/rfc4954
- RFC 3501 (IMAP4rev1 — LOGIN/LIST/SELECT/FETCH): https://www.rfc-editor.org/rfc/rfc3501
- RFC 1939 (POP3 — USER/PASS/RETR): https://www.rfc-editor.org/rfc/rfc1939
- Project Zero — Exploiting the Linux kernel via packet sockets (AF_PACKET needs CAP_NET_RAW, obtainable in a userns): https://projectzero.google/2017/05/exploiting-linux-kernel-via-packet.html
- man7 raw(7) / packet(7) — raw + AF_PACKET socket semantics: https://www.man7.org/linux/man-pages/man7/raw.7.html
- mwalle/rawsocket — raw sockets as an unprivileged user (userns): https://github.com/mwalle/rawsocket
- nickb.dev — Viewing WireGuard traffic with tcpdump (encrypted on the wire; `0x04` data-message prefix): https://nickb.dev/blog/viewing-wireguard-traffic-with-tcpdump/
- Pro Custodibus — Troubleshooting WireGuard with tcpdump (capture underlay vs `wg0`): https://www.procustodibus.com/blog/2023/05/troubleshooting-wireguard-with-tcpdump/
- IVPN — Self-audit your VPN Pt2 (confirm no plaintext leaks on the physical interface): https://www.ivpn.net/privacy-guides/self-audit-series-part2/
- Podman docs — podman-unshare: https://docs.podman.io/en/latest/markdown/podman-unshare.1.html
- slirp4netns: https://github.com/rootless-containers/slirp4netns

- passt/pasta: https://passt.top/passt/about/
- sanj.dev — Podman pasta vs slirp4netns: https://sanj.dev/post/podman-pasta-vs-slirp4netns-networking/
- Cloudflare — BoringTun userspace WireGuard: https://blog.cloudflare.com/boringtun-userspace-wireguard-rust/
- cloudflare/boringtun: https://github.com/cloudflare/boringtun
- cmusatyalab/wireguard4netns: https://github.com/cmusatyalab/wireguard4netns
- gVisor `pkg/tcpip/stack` GoDoc: https://pkg.go.dev/gvisor.dev/gvisor/pkg/tcpip/stack
- ryan-schachte — userspace WireGuard with netstack: https://ryan-schachte.com/blog/userspace_wireguard_tunnels/
- python-zeroconf (pure-python mDNS): https://github.com/python-zeroconf/python-zeroconf
- Wikipedia — Metamorphic testing: https://en.wikipedia.org/wiki/Metamorphic_testing
- emergentmind — Metamorphic Relations: https://www.emergentmind.com/topics/metamorphic-relations-mrs
- arXiv 1912.05278 — Metamorphic Security Testing for Web Systems: https://arxiv.org/pdf/1912.05278